Recitation 5

CS211 Sections 10-13

To do

- Signed number representation
- Binary arithmetic
- overflow

Introduction

```c
#include<stdio.h>
int main(int argc, const char** argv){
    printf("hello!\n");
    return -1;
}
```

after running this program, then writing `echo $?`, what do we get?

Introduction

```c
#include<stdio.h>
int main(int argc, const char** argv){
    printf("hello!\n");
    return -1;
}
```

after running this program, then writing `echo $?`, what do we get? We'd get 255. Why is this?

Review

Convert the following to binary, hex, and octal decimal: 194

Answer

Convert the following: 194

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
128	64	32	16	8	4	2	1

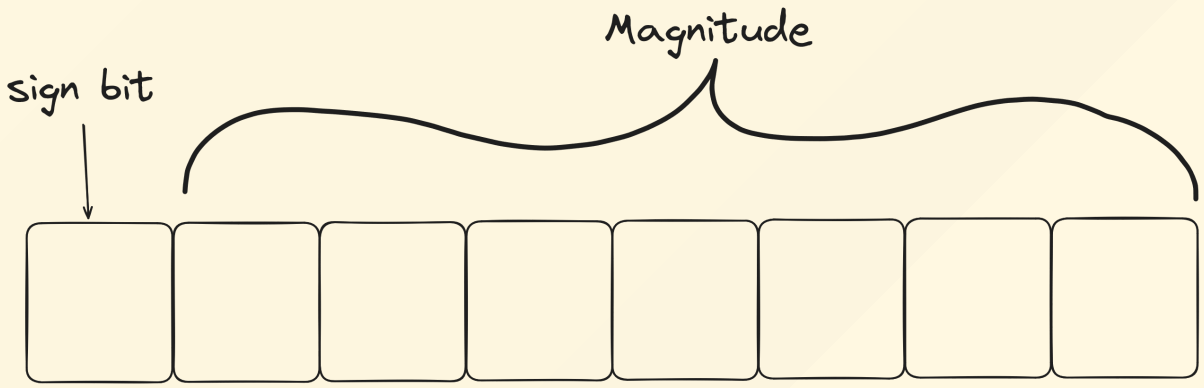
$194 - 128 = 66 - 64 = 2 - 2 = 0$
So, we can say $194 = 128 + 64 + 4 = 2^7 + 2^6 + 2^1$
So, $194_{10} = 11000010_2$
Convert to hex: 1100 0010, 1100 = C, 0010 = 2
194 = $C2_{16}$
octal: $011\ 000\ 010 = 302_8$

Signed vs. Unsigned

- What's the difference between the two?
- How do we represent positive and negative numbers in decimals?
- How do we represent it in binary? We have two options:

Signed magnitude

- One way to represent numbers is by using the Most Significant bit as a sign
  - 1 for negative, 0 for positive.



- This has problems: how do we represent 0?

Ones complement

- Another way to represent signed numbers is by using ones complement.
- You simply flip the bits in a number:
  - 1 becomes 0
  - 0 becomes 1

example: 00000101 in ones complement is 11111010

Twos Complement

- This is the more common approach to represent signed integers
- The Most significant bit will be -2^{k-1}, for a k bit signed data.
  - For example, for an 8-bit signed integer, -128 will be the most significant bit.

example

$$-1 = -128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = -128 + 127 = -1$$

- How to get twos complement of a number?

Binary arithmetic - Addition

- How do we add binary numbers?
  - There are four basic rules for addition:
    - $0 + 0 = 0$
    - $0 + 1 = 1$
    - $1 + 0 = 1$
    - $1 + 1 = 10$
      - Here, we carry the one to the next spot.
- let's do an example, $1011_2 + 1110_2$

Subtraction with twos complement

- Let's say we're given 50 - 20; what's another way of seeing this?
  - $50 + (-20)$
  - We can express -20 in two complements, then add the 50 and -20 together.

Subtraction with ones compliment

- Very similar to the previous problem, we can express 50 - 20 as $50 + (-20)$
  - flip the bits of the negative number
  - Do the addition as usual
  - if you get a carry at the end, you add it to the final sum, this is end by carry.
  - if you don't, you do one's complement of the sum, and you add a negative sign.
- You can see, this is very involved.

Multiplication

- Very similar to multiplication in decimal.

let's find $01011010_2 \times 00000011_2$

Division

similar to division in decimal

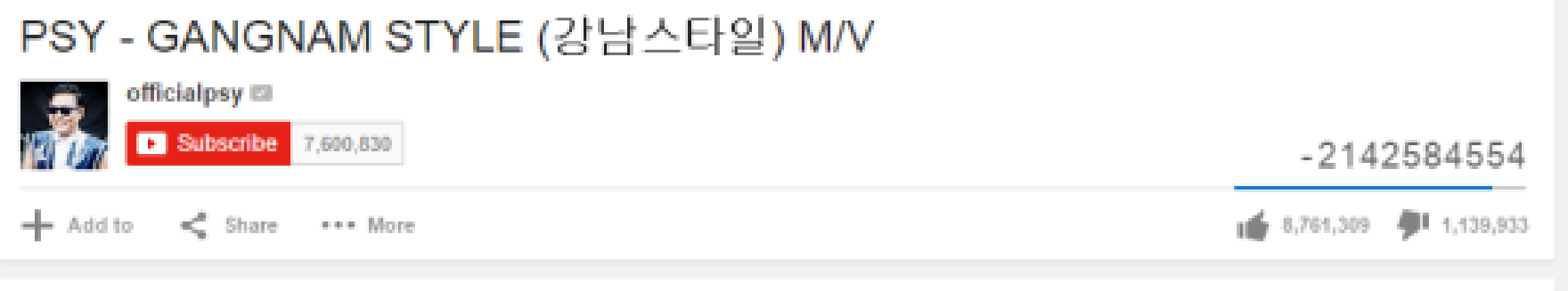
let's find $110_2 \div 010_2$

Practice

find the sum of $01111111_2 + 1_2$

Overflow

- We know data types can only hold so much; what happens when we exceed the limit?



- YouTube used a 32-bit integer for the view counter. Now, they use a 64-bit counter

What happens when you overflow?

- Let's say you have an unsigned 8-bit integer
  - the max value is 255_{10}, or 11111111_2
  - let's add one, which is 256_{10}, or 100000000_2.
    - We only look at the 8 bits $1 \mid 00000000 \mid$, so 256 would wrap back to zero.

- This is an illustration of how overflow work

Questions

- Why is twos complement for subtraction preferred over ones complement?
- What is the issue with signed magnitude?